

app\resistance_simulation.py

```
1  # -*- coding: utf-8 -*-
2  """
3  @file      resistance_simulation.py
4  @brief     Module pour commander le MUX de simulation de résistance.
5  @details   Ce module permet d'appliquer une résistance simulée R_sim via
6             le réseau de résistances contrôlé par multiplexeur ADG731.
7             Il gère le calcul des commutateurs à activer, la commande du
8             multiplexeur, et la validation des valeurs appliquées.
9  @author    Léo Mendes
10 @project    2510_RegulationsChauffage
11 @Mandant    Oblo_solution
12 @date       2025-09-18
13 @version    1.0.0
14 @copyright  Copyright (c) 2025
15 """
16
17 # Import des modules système standard
18 import time
19 # Import des annotations de type pour la documentation
20 from typing import Optional, List, Dict, Tuple
21 # Import du driver multiplexeur ADG731
22 from app.hw_mux_adg731 import Adg731MuxSpi
23 # Import des utilitaires de mapping de résistances
24 from app.resistance_mapping import get_channel_mux_map, find_closest_channel
25
26
27 # -----
28 # Classes d'exception personnalisées
29 # -----
30
31 class ResistanceSimulationError(Exception):
32     """
33     @brief   Exception pour les erreurs de simulation de résistance.
34     @details Exception spécialisée levée lors d'erreurs dans le processus
35             de simulation de résistance via le multiplexeur.
36     """
37     pass
38
39
40 # -----
41 # Classe principale de simulation
42 # -----
43
44 class ResistanceSimulator:
45     """
46     @brief   Contrôleur pour la simulation de résistance via MUX.
47     @details Classe principale qui encapsule toute la logique de simulation
48             de résistance en utilisant le multiplexeur ADG731 et les
49             mappings de résistances calibrées.
50     """
51
52     def __init__(self, speed_hz: int = 100000):
53         """
54         @brief   Initialise le simulateur de résistance.
55         @details Constructeur qui configure le multiplexeur ADG731 avec
56             la vitesse SPI spécifiée et initialise les variables
57             d'état du simulateur.
58
59         @param speed_hz  Vitesse de communication SPI en Hz (défaut: 100kHz).
60
61         @exception ResistanceSimulationError si l'initialisation du MUX échoue.
62         """
63         # Référence vers le multiplexeur ADG731
64         self.mux = None
65         # Vitesse de communication SPI configurée
66         self.speed_hz = speed_hz
67         # Résistance actuellement appliquée (None si aucune)
68         self.current_resistance = None
69         # Liste des canaux MUX actuellement actifs
70         self.active_channels = []
71
72         try:
73             # Initialisation du multiplexeur ADG731 avec la vitesse SPI
74             self.mux = Adg731MuxSpi(speed_hz=speed_hz)
75             # Affichage de confirmation d'initialisation
76             print(f"[SIM_R] Simulateur de résistance initialisé (SPI: {speed_hz} Hz)")
77         except Exception as e:
78             # Affichage d'erreur en cas d'échec d'initialisation
79             print(f"[SIM_R] ERREUR d'initialisation du MUX: {e}")
80             # Levée d'exception avec contexte détaillé
81             raise ResistanceSimulationError(f"Impossible d'initialiser le MUX: {e}")
82
83     def close(self):
84         """
```

```
85     @brief Ferme les connexions du simulateur.
86     @details Méthode de nettoyage qui ferme proprement la connexion
87             SPI avec le multiplexeur et libère les ressources.
88     """
89     # Vérification de l'existence de la connexion MUX
90     if self.mux:
91         try:
92             # Fermeture de la connexion SPI
93             self.mux.close()
94             # Confirmation de fermeture
95             print("[SIM_R] Simulateur fermé")
96         except Exception as e:
97             # Affichage d'erreur en cas de problème de fermeture
98             print(f"[SIM_R] Erreur lors de la fermeture: {e}")
99
100 def calculate_parallel_resistance(self, resistances: List[float]) -> float:
101     """
102     @brief Calcule la résistance équivalente de résistances en parallèle.
103     @details Applique la formule  $1/R_{eq} = 1/R_1 + 1/R_2 + \dots + 1/R_n$ 
104             pour calculer la résistance équivalente d'un ensemble
105             de résistances connectées en parallèle.
106
107     @param resistances Liste des valeurs de résistance en parallèle [Ohms].
108
109     @return Résistance équivalente calculée [Ohms].
110             Retourne l'infini si la liste est vide.
111     """
112     # Cas d'une liste vide: résistance infinie
113     if not resistances:
114         return float('inf')
115
116     # Cas d'une seule résistance: pas de calcul parallèle
117     if len(resistances) == 1:
118         return resistances[0]
119
120     # Calcul de la somme des inverses ( $1/R_1 + 1/R_2 + \dots$ )
121     reciprocal_sum = sum(1.0 / r for r in resistances)
122     # Application de la formule:  $R_{eq} = 1 / (\text{somme des } 1/R_i)$ 
123     return 1.0 / reciprocal_sum
124
125 def apply_resistance_simulation(self, target_resistance: float, measurement_channel: int, sensor_type: str = "Ni1000 TK5000") -> bool:
126     """
127     @brief Applique une résistance simulée via le MUX sur le canal de mesure.
128     @details Fonction principale qui recherche le canal MUX optimal pour
129             simuler la résistance cible, configure le multiplexeur et
130             mémorise l'état de la simulation.
131
132     @param target_resistance Résistance à simuler [Ohms].
133     @param measurement_channel Canal sur lequel la mesure a été faite (info).
134     @param sensor_type Type de sonde pour sélectionner la carte MUX.
135
136     @return True si succès, False en cas d'erreur.
137     """
138     # Affichage de l'en-tête de la procédure de simulation
139     print(f"\n[SIM_R] === APPLICATION RÉSISTANCE SIMULÉE ===")
140     # Affichage de la résistance cible avec précision
141     print(f"[SIM_R] Résistance cible: {target_resistance:.2f} Ω")
142     # Affichage du canal de mesure pour traçabilité
143     print(f"[SIM_R] Canal de mesure: {measurement_channel}")
144     # Affichage du type de sonde sélectionné
145     print(f"[SIM_R] Type de sonde: {sensor_type}")
146
147     # Vérification de l'initialisation du multiplexeur
148     if self.mux is None:
149         # Erreur critique: MUX non initialisé
150         print("[SIM_R] ERREUR MUX non initialisé")
151         return False
152
153     # Recherche du canal MUX optimal en utilisant le mapping dynamique
154     try:
155         # Appel de la fonction de recherche du canal le plus proche
156         channel, actual_resistance, error_percent = find_closest_channel(target_resistance, sensor_type)
157
158         # Vérification que la recherche a trouvé un canal valide
159         if channel is None:
160             # Aucun canal MUX disponible pour cette résistance
161             print(f"[SIM_R] ERREUR Aucun canal MUX trouvé pour {target_resistance:.1f}Ω")
162             return False
163
164         # Affichage du canal MUX sélectionné
165         print(f"[SIM_R] Canal MUX trouvé: {channel}")
166         # Affichage de la résistance réelle vs cible
167         print(f"[SIM_R] Résistance MUX: {actual_resistance}Ω (cible: {target_resistance:.1f}Ω)")
168         # Affichage de l'erreur relative en pourcentage
169         print(f"[SIM_R] Erreur: {error_percent:.1f}%")
170
171     except KeyError as e:
```

```
172         # Erreur: type de sonde non supporté
173         print(f"[SIM_R] ERREUR {e}")
174         return False
175
176     try:
177         # Configuration du multiplexeur (toujours carte 0 par défaut)
178         board_index = 0
179         # Affichage de l'action de configuration MUX
180         print(f"[SIM_R] Application MUX canal {channel} (carte {board_index})")
181         # Activation du canal sélectionné sur la carte MUX
182         self.mux.set_channel(board_index, channel)
183
184         # Mémorisation de l'état actuel de simulation
185         self.active_channels = [channel]
186         # Sauvegarde de la résistance réellement appliquée
187         self.current_resistance = actual_resistance
188
189         # Confirmation de succès avec résistance appliquée
190         print(f"[SIM_R] SUCCES Résistance appliquée: {actual_resistance}Ω sur canal MUX {channel}")
191         return True
192
193     except Exception as e:
194         # Erreur lors de la communication avec le MUX
195         print(f"[SIM_R] ERREUR lors de l'application: {e}")
196         return False
197
198     def get_current_simulation(self) -> Dict[str, any]:
199         """
200         @brief Retourne l'état actuel de la simulation.
201         @details Fonction d'interrogation qui fournit un dictionnaire
202                 contenant toutes les informations sur l'état actuel
203                 de la simulation de résistance.
204
205         @return Dictionnaire contenant les informations actuelles:
206                 - resistance: valeur appliquée [Ohms] ou None
207                 - active_channels: liste des canaux MUX actifs
208                 - mux_available: disponibilité du multiplexeur
209         """
210         # Construction du dictionnaire d'état avec résistance actuelle
211         return {
212             "resistance": self.current_resistance,
213             # Copie de la liste des canaux actifs (protection contre modification)
214             "active_channels": self.active_channels.copy() if self.active_channels else [],
215             # Indication de disponibilité du MUX (toujours True si initialisé)
216             "mux_available": True
217         }
218
219     def validate_resistance_applied(self, expected_resistance: float, tolerance: float = 0.10) -> bool:
220         """
221         @brief Valide que la résistance appliquée correspond à l'attendu.
222         @details Fonction de validation qui compare la résistance actuellement
223                 appliquée avec la valeur attendue en tenant compte d'une
224                 tolérance relative spécifiée.
225
226         @param expected_resistance Résistance attendue pour la validation [Ohms].
227         @param tolerance Tolérance relative acceptée (défaut: 10%).
228
229         @return True si la résistance est dans la tolérance.
230         """
231         # Vérification qu'une résistance est actuellement appliquée
232         if self.current_resistance is None:
233             # Erreur: aucune résistance n'est actuellement simulée
234             print(f"[SIM_R] ERREUR Aucune résistance actuellement appliquée")
235             return False
236
237         # Calcul de l'erreur relative entre résistance appliquée et attendue
238         error = abs(self.current_resistance - expected_resistance) / expected_resistance
239
240         # Comparaison de l'erreur avec la tolérance spécifiée
241         if error <= tolerance:
242             # Validation réussie: affichage des détails
243             print(f"[SIM_R] VALIDATION OK: {self.current_resistance:.2f}Ω (attendu: {expected_resistance:.2f}Ω, erreur: {error*100:.1f}%)")
244             return True
245         else:
246             # Validation échouée: affichage des détails d'erreur
247             print(f"[SIM_R] VALIDATION KO: {self.current_resistance:.2f}Ω (attendu: {expected_resistance:.2f}Ω, erreur: {error*100:.1f}%)")
248             return False
249
250     def reset_simulation(self):
251         """
252         @brief Remet la simulation à zéro (désactive tous les canaux).
253         @details Procédure de nettoyage qui désactive tous les canaux de
254                 tous les multiplexeurs et remet l'état de simulation
255                 à sa configuration initiale.
256         """
257         # Affichage du début de la procédure de remise à zéro
258         print("[SIM_R] Remise à zéro de la simulation")
```

```
259
260     # Vérification de l'initialisation du multiplexeur
261     if self.mux is None:
262         return
263
264     try:
265         # Désactivation systématique de tous les canaux
266         for board in range(4):
267             # Parcours de tous les canaux de chaque carte
268             for channel in range(32):
269                 try:
270                     # Tentative de désactivation du canal
271                     self.mux.set_channel(board, channel)
272                 except Exception:
273                     # Ignore les erreurs individuelles de canal
274                     pass
275
276         # Remise à zéro des variables d'état
277         self.active_channels = []
278         # Effacement de la résistance mémorisée
279         self.current_resistance = None
280         # Confirmation de succès de la remise à zéro
281         print("[SIM_R] SUCCES Simulation remise à zéro")
282
283     except Exception as e:
284         # Erreur lors de la procédure de remise à zéro
285         print(f"[SIM_R] ERREUR lors de la remise à zéro: {e}")
286
287
288 # -----
289 # Fonctions utilitaires globales
290 # -----
291
292 def apply_resistance_simulation(resistance_ohm: float) -> bool:
293     """
294     @brief   Fonction utilitaire pour appliquer une résistance simulée.
295     @details Fonction de commodité qui encapsule la création d'un simulateur,
296             l'application d'une résistance et le nettoyage automatique.
297             Utilise le pattern RAII pour la gestion des ressources.
298
299     @param resistance_ohm  Résistance à appliquer [Ohms].
300
301     @return                True si l'application a réussi, False sinon.
302     """
303     # Initialisation du simulateur à None pour le nettoyage
304     simulator = None
305     try:
306         # Création d'une instance du simulateur
307         simulator = ResistanceSimulator()
308         # Application de la résistance avec paramètres par défaut
309         return simulator.apply_resistance_simulation(resistance_ohm)
310     except Exception as e:
311         # Gestion d'erreur avec affichage du contexte
312         print(f"[SIM_R] ERREUR dans apply_resistance_simulation: {e}")
313         return False
314     finally:
315         # Nettoyage automatique du simulateur si créé
316         if simulator:
317             simulator.close()
318
319
320 # -----
321 # Section de test et démonstration
322 # -----
323 if __name__ == "__main__":
324     """
325     @brief   Point d'entrée pour les tests de développement.
326     @details Section de test qui démontre l'utilisation du simulateur
327             avec différents cas d'usage et validation des résultats.
328     """
329     # Affichage de l'en-tête des tests de développement
330     print("=== Test du simulateur de résistance ===")
331
332     # Création d'une instance de simulateur pour les tests
333     sim = ResistanceSimulator()
334
335     # Test 1: Application d'une résistance simple de 1000 Ohms
336     print("\nTest 1: Application de 1000Ω")
337     # Appel de la fonction de simulation avec résistance cible
338     success = sim.apply_resistance_simulation(1000.0)
339     # Affichage du résultat du test avec évaluation booléenne
340     print(f"Résultat: {'Succès' if success else 'Échec'}")
341
342     # Test 2: Application d'une résistance complexe (parallèle calculé)
343     print("\nTest 2: Application de 667Ω (parallèle de 1k et 2k)")
344     # Test avec une valeur nécessitant une approximation par le MUX
345     success = sim.apply_resistance_simulation(667.0)
```

```
346 # Évaluation et affichage du résultat
347 print(f"Résultat: {'Succès' if success else 'Échec'}")
348
349 # Test 3: Validation de la résistance appliquée avec tolérance
350 print("\nTest 3: Validation de la résistance appliquée")
351 # Vérification qu'une résistance est effectivement appliquée
352 if sim.current_resistance:
353     # Validation avec tolérance élargie de 15%
354     valid = sim.validate_resistance_applied(667.0, tolerance=0.15)
355     # Affichage du résultat de validation
356     print(f"Validation: {'OK' if valid else 'KO'}")
357
358 # Procédure de nettoyage et fermeture
359 sim.reset_simulation()
360 # Fermeture propre du simulateur
361 sim.close()
```